

OPTIMIZACIÓN DEL COMPILADOR CIRCOM OPTIMIZATION OF THE CIRCOM COMPILER



TRABAJO FIN DE GRADO
CURSO 2025-2026

AUTHOR
PEDRO TOBALINA NAVARRO

DIRECTOR
CLARA RODRÍGUEZ NÚÑEZ

GRADO EN INGENIERÍA INFORMÁTICA
FACULTAD DE INFORMÁTICA
UNIVERSIDAD COMPLUTENSE DE MADRID

OPTIMIZACION DEL COMPILADOR CIRCOM OPTIMIZATION OF THE CIRCOM COMPILER

TRABAJO DE FIN DE GRADO EN INGENIERÍA INFORMÁTICA

AUTHOR

PEDRO TOBALINA NAVARRO

DIRECTOR

CLARA RODRÍGUEZ NÚÑEZ

CONVOCATORIA: SEPTIEMBRE 2026

GRADO EN INGENIERÍA INFORMÁTICA
FACULTAD DE INFORMÁTICA
UNIVERSIDAD COMPLUTENSE DE MADRID

30 DE AGOSTO DE 2026

DEDICATORY

To all the teachers for the time they spend
teaching new generations.

ACKNOWLEDGMENTS

To Clara, she has dedicated herself to helping me with this project all the time.
And for all the teachers that have helped me reach this point in life.

ABSTRACT

Optimization of the Circom compiler

Circom is a domain-specific language widely used for describing arithmetic circuits in zk-SNARK systems. By default, the Circom compiler represents all signals and variables as elements of a large finite field, utilizing a prime number modulus of up to 256 bits (32 bytes). This generic representation introduces inefficiencies in both C++ and Web Assembly generated code, as some intermediate circuit values only require much smaller data types, such as booleans (1 byte) or standard integers (4 bytes).

This project proposes an optimization on the Rust-based Circom compiler by introducing a bounds analysis phase. By evaluating operations, expressions, conditionals, and bounds inheritance across templates, the compiler calculates minimum and maximum ranges for every signal. Variables with restricted ranges are decreased to smaller memory footprints. Testing the optimized compiler against the standard Circomlib repository demonstrated an average memory usage reduction of 30.4%. This smaller memory representation directly improves the execution speed and efficiency of the generated witness calculator without altering the mathematical correctness of the zero-knowledge proofs.

Keywords

Circom, Compiler Optimization, zk-SNARKs, Memory Efficiency, Bounds Analysis, Rust.

TABLE OF CONTENTS

Capítulo 1 - Introduction	13
1.1 Motivations	14
1.2 Circom	14
1.3 Rust	16
1.4 Objectives	17
1.5 Work Plan	17
Capítulo 2 - The problem to solve	18
Capítulo 3 - Analysis	21
3.1 Compute bounds	21
3.1.1 Infix operations	22
3.1.2 Prefix operations	25
3.1.3 Switch operations	26
3.1.4 Variables	26
3.1.5 Numbers	26
3.1.6 Array in line	26
3.1.7 Uniform array	27
3.2 Treat statements	27
3.2.1 Treat conditionals	27
3.2.2 Treat while loops	28
3.3 Bounds inheritance	29
Capítulo 4 - Testing	31
Capítulo 5 - Conclusions and future work	33
Bibliography	35

LIST OF FIGURES

Figure 3-1. Example from Circomlib	14
Figure 3-2. IsZero template From Circomlib	18

LIST OF TABLES

Table 3-1. Test results.....	32
------------------------------	----

Capítulo 1 - Introduction

Zero-knowledge cryptography allows one party to prove that a statement is true without revealing any additional information beyond the validity of the statement itself. In this context, zk-SNARKs are one of the most widely used constructions, since they provide short proofs that can be verified efficiently. To generate the constraint systems required by these protocols, several domain-specific languages have been developed. These languages make it easier for users to describe arithmetic circuits and other cryptographic computations in a high-level way, without having to deal directly with the low-level representation of constraints.

Among these languages, Circom stands out as one of the most relevant and widely used. It was originally developed at the Universidad Complutense de Madrid in collaboration with the company iden3. Circom is currently used by tens of thousands of users in both academic and practical contexts, and it is taught in several universities as part of applied cryptography courses. It is also widely used in privacy-oriented applications such as digital identity, voting systems, and blockchain-based solutions. In these scenarios, the efficiency of the compiler becomes critical, especially when working with large circuits, since both memory usage and execution time can significantly affect performance. Circom not only generates the constraint system of the circuit but also produces executable code in both C++ and Web Assembly to compute the witness.

The Circom compiler is implemented in Rust, a systems programming language that combines high performance with memory safety. This makes Rust a suitable choice for a compiler, where large data structures and complex transformations must be handled efficiently and reliably. In addition, Rust provides fine grained control over memory representation, which is especially valuable in this project, since improving memory usage can improve both the compiler and the generated executable code. Smaller internal representations can reduce memory consumption and improve execution time, because operations over smaller values are generally easier and faster to process.

The objective of this work is to analyze the range of values that variables can take and, depending on those bounds, adjust their memory size, accordingly, reducing overall memory usage without affecting correctness.

1.1 Motivations

The main motivation for my TFG is to address a real-world problem using a tool already employed in academic and industrial settings. Circom is an important compiler within the zero-knowledge proofing ecosystem, so any improvement in its efficiency could have immediate practical applications. Optimizing memory usage is important because it directly impacts the generated code, which will require less memory usage and will run faster. Furthermore, the project allows me to work with two particularly appealing areas of computer science: compilers and Rust programming.

1.2 Circom

Circom is a **Domain-Specific Language** (DSL) for describing arithmetic circuits used in zk-SNARK systems. Circuits consist of addition and multiplication gates over a finite field. A typical Circom program defines signals, components, and templates in a C-like syntax. Circom also supports arrays of signals, loops, and user-defined templates, enabling modular circuit design. This modularity makes it easier to compose and audit large circuits: each template can be tested independently before instantiation.

```
template Montgomery2Edwards() {  
    signal input in[2];  
    signal output out[2];  
  
    out[0] <-- in[0] / in[1];  
    out[1] <-- (in[0] - 1) / (in[0] + 1);  
  
    out[0] * in[1] === in[0];  
    out[1] * (in[0] + 1) === in[0] - 1;  
}  
  
template AND() {  
    signal input a;  
    signal input b;  
    signal output out;  
  
    out <== a*b;  
}
```

Figure 1-1. Example from Circomlib

In those examples we can see two templates from Circom lib. We can observe how Circom is written, and the different ways to assign values to signals and create constraints.

"<==" is used to assign a value and create a constraint.

"<--" is used to only assign a value.

"===" is used to only create a constraint.

When a Circom ". Circom" file is compiled, the Circom compiler performs several phases. It parses the source into an **Abstract Syntax Tree** (AST), performs semantic analysis, and then synthesizes constraints. Every arithmetic operation in the circuit (such as "out <== x * y") is translated into one or more rank-1 constraints of the form $(\text{linear_combination_A}) * (\text{linear_combination_B}) = (\text{linear_combination_C})$.

The Circom compiler produces two main outputs:

- An .r1cs file, which contains the full rank-1 constraint system that encodes the circuit.
- A witness calculator program, generated in C++ or Web Assembly, which computes all the signal values that satisfy those constraints.

The .r1cs file stores the constraints derived from the circuit. For example, in the AND template, the statement `out <== a * b;` generates both the assignment and the corresponding constraint that enforces `out = a * b`. In the Edwards2Montgomery template, the lines with `===` generate explicit constraints, while the lines with `<--` only compute intermediate values without adding constraints. In this case, the witness calculator uses the assignments to compute the values of `out [0]` and `out [1]`, and the .r1cs file stores the equations that those values must satisfy.

The witness calculator is responsible for computing all intermediate and final signal values from the circuit inputs. Once this witness is generated, it can be combined with the .r1cs file and passed to tools such as SnarkJS, which are used to generate and verify zero-knowledge proofs. In this way, Circom separates the description of the arithmetic circuit from the cryptographic machinery required to produce proofs.

1.3 Rust

Rust is a modern systems programming language designed to provide high performance while ensuring memory safety. Unlike traditional languages such as C++, Rust enforces strict compile-time guarantees through its ownership and borrowing model, preventing common errors such as null pointer dereferencing, data races, and memory leaks.

Rust's safety and performance make it well-suited for a compiler managing large data structures (like constraint matrices). Some of Rust advantages:

- **Memory Safety:** Rust enforces memory safety at compile time, preventing use-after-free, buffer overruns, and data races, without any garbage collector overhead. This means the compiler avoids a whole class of bugs, increasing reliability.
- **Zero-Cost Abstractions:** Rust's abstractions (e.g. generics, iterators) compile down to efficient code. The compiler itself can be written using high-level constructs, but at run-time there is no hidden cost. This means developers of Circom can use powerful data structures and strong typing without sacrificing speed.
- **Strong Type System:** Rust's rich type system aids in writing compiler code. For example, signals, components, and constraints can be modeled with Rust enums and checked exhaustively by the compiler. This reduces bugs in handling complex cases.

1.4 Objectives

The overall objective of this TFG is to analyze the memory usage behavior of the Circom compiler and propose an improvement to the representation of its internal variables. To achieve it, the different types of operations and statements in the Circom language will be studied to find the range of values that signals and variables can take. Based on this analysis, an attempt will be made to replace generic and oversized representations with others better suited to the actual needs of each case. The aim is to reduce the memory sizes in the generated code without compromising the correctness of the compilation process.

1.5 Work Plan

The work is carried out in several phases.

First, it is necessary to become familiar with Rust and the fundamentals of the Circom compiler, as well as how circuits and constraints are represented.

The next stage involves analyzing the source code to identify areas where memory usage could be optimized.

Finally, the necessary modifications are implemented, and their impact on the compiler's behavior is evaluated.

Capítulo 2 - The problem to solve

Unlike many other compilers, Circom does not work with several primitive variable types. Instead, its signals are elements of a finite field. In practice, these fields are very large, and their modulus is usually a prime number with a size of at most 256 bits. This means that every signal is, by default, represented using a much larger range than most circuit values require.

This also creates inefficiency not only when generating C code, but also when targeting Web Assembly. Since all values are treated as large field elements, the generated code must handle them using representations that are often larger and more expensive than necessary. As a result, memory consumption and execution time can increase, especially in circuits with many intermediate values.

A simple example of this situation is the "IsZero" template. In this circuit, the output signal can only take two possible values, 0 or 1. Even so, without any additional analysis, it is still represented as a full field element. If the compiler can find that the signal is Boolean, it can use that information to reduce the amount of memory needed, and to generate more efficient code.

```
template IsZero() {  
    signal input in;  
    signal output out;  
  
    signal inv;  
  
    inv <-- in!=0 ? 1/in : 0;  
  
    out <== -in*inv +1;  
    in*out === 0;  
}
```

Figure 2-1. IsZero template From Circomlib

The work presented in this project is divided into two main phases. First, the bounds of the different signals are detected by analyzing the operations and statements that define them. Once these bounds are known, they can be used in two ways: to generate more efficient C or Web Assembly code, and to verify that the constraints produced by the circuit are sufficient to guarantee that those bounds always hold.

Capítulo 3 - Analysis

This chapter describes the methodology used to analyze the size requirements of the variables and signals handled by the compiler. The goal of this analysis is to traverse the different instructions of a program and find the range of values that each signal can take. Once these bounds are known, they can be used to reduce the memory representation of variables, when possible, without changing the correctness of the compilation process.

Since Circom operates over a finite field, every value is always within the range $[0, p)$, where p is a large prime number. This range acts as a general upper bound for all expressions, but in many cases the actual values are much smaller. The purpose of the analysis is to detect these tighter bounds and exploit them when generating more efficient representations.

The analysis is divided into two main tasks. First, we need to compute the bounds of expressions, which is presented in Section 3.1. Second, we need to update and maintain the current state of the known bounds during the traversal of statements, which is described in Section 3.2. To support this process, the compiler stores a context structure in the form of a hash map, where each signal name is associated with its founded bounds. This allows the compiler to reuse previously computed information when processing statements, instead of recomputing the bounds of the same signal every time they are needed.

3.1 Compute bounds

The computing of bounds is a very sophisticated part of the program, in which you get an expression, and depending on the kind of the expression, you must carefully analyze what the minimum and maximum bounds are.

To do that I am going to implement a function that given the expression and the context, will give back the bounds.

$$\text{Bounds}(\text{Expr}, \text{context}) = \{\text{min}, \text{max}\}$$

3.1.1 Infix operations

In this section, we consider the infix operators, that is, operations written between two expressions. These include arithmetic operators such as addition, subtraction, and multiplication, as well as logical and bitwise operators. Since each of these operators combines two operands, the first step is to compute the bounds of both expressions. Then, depending on the operator, the resulting bounds are calculated using the corresponding rule.

3.1.1.1 Arithmetic integer operations

These 3 operations (Add, Sub, and Mul) are very similar among them in the way their bounds are calculated.

For the Add operation you just have to add the bounds, min with min and max with max, and you will have the bounds of the whole expression.

For the Sub operation is a bit different but the same concept, you want the minimum to be the smallest possible result, so you subtract to the left min the right max, and with the maximum the other way around, subtract to the left max the right min.

And finally, the Mul operations works as the Add operation with the twist that now you multiply instead of summing.

If $\text{Bounds}(e1) = (\min1, \max1)$ and $\text{Bounds}(e2) = (\min2, \max2)$ then

$\text{Bounds}(e1 + e2) = (\min1 + \min2, \max1 + \max2)$

$\text{Bounds}(e1 - e2) = (\min1 - \max2, \max1 - \min2)$

$\text{Bounds}(e1 * e2) = (\min1 * \min2, \max1 * \max2)$

As an example, suppose that $x0 \in [0, 5]$ and $x1 \in [2, 3]$. Then:

$\text{Bounds}(x0 + x1) = (0 + 2, 5 + 3) = (2, 8)$

$\text{Bounds}(x0 - x1) = (0 - 3, 5 - 2) = (-3, 3)$

$\text{Bounds}(x0 \cdot x1) = (0 \cdot 2, 5 \cdot 3) = (0, 15)$

3.1.1.2 Division operations

There are two kinds of divisions in Circom, the Division and the Integer Division.

For the Division Since the quotient may vary widely, the analysis uses the most conservative bound and assumes that the result can range over the whole field.

For integer division, we can at least define the maximum value, and that because in the left operand we cannot have a number below 1:

If $\text{Bounds}(e1) = (\min1, \max1)$ and $\text{Bounds}(e2) = (\min2, \max2)$ then

$\text{Bounds}(e1 / e2) = (0, \text{prime})$

$\text{Bounds}(e1 \setminus e2) = (0, \max1/\min2)$

3.1.1.3 Power and modulus operations

These operations have very easy bounds. For the power, we are going to keep the prime as maximum, and 0 for the minimum. And for the modulus, the minimum would be 0, and the maximum will be the expression on the right.

If $\text{Bounds}(e1) = (\min1, \max1)$ and $\text{Bounds}(e2) = (\min2, \max2)$ then

$\text{Bounds}(e1 \% e2) = (0, \max2)$

$\text{Bounds}(e1 ** e2) = (0, \text{prime})$

3.1.1.4 Shift operations

For the shift left, the maximum and minimum will be shifted so we shall multiply them by 2 to the power of the minimum and maximum. As for the right shift, we will just keep the maximum of the left expression, and for the minimum we will have a 0.

If $\text{Bounds}(e1) = (\min1, \max1)$ and $\text{Bounds}(e2) = (\min2, \max2)$ then

$\text{Bounds}(e1 \ll e2) = (\min1 * 2^{\min2}, \max1 * 2^{\max2})$

$\text{Bounds}(e1 \gg e2) = (0, \max1)$

3.1.1.5 Boolean operations

The operators considered in this group are Lesser, LesserEq, Greater, GreaterEq, Eq, NotEq, BoolOr, and BoolAnd. Their common characteristic is that they always produce a boolean result, that is, either 0 or 1. Because of that, the bounds of any expression using one of these operators are always:

If $\text{Bounds}(e1) = (\min1, \max1)$ and $\text{Bounds}(e2) = (\min2, \max2)$ then

$\text{Bounds}(e1 \text{ "Boolean operation" } e2) = (0, 1)$

3.1.1.6 Bit operations

For the "Or" operation, we just must add the maximus and find the biggest minimum of both expressions. In the other hand, the "And" operation keeps a 0 in its minimum, and the minimum max of both expressions as the maximus. And finally, the "XOr" operation has the minimum of the "And", and the maximum of the "Or".

If $\text{Bounds}(e1) = (\min1, \max1)$ and $\text{Bounds}(e2) = (\min2, \max2)$ then

$\text{Bounds}(e1 \mid e2) = \{\max(\min1, \min2), \max1 + \max2\}$

$\text{Bounds}(e1 \& e2) = \{0, \min(\max1, \max2)\}$

$\text{Bounds}(e1 \wedge e2) = \{0, \max1 + \max2\}$

As an example, suppose that $x0 \in [0, 5]$ and $x1 \in [2, 3]$. Then:

$\text{Bounds}(x0 \mid x1) = \{\max(0, 2), 5 + 3\} = (2, 8)$

$\text{Bounds}(x0 \& x1) = \{0, \min(5, 3)\} = (0, 3)$

$\text{Bounds}(x0 \wedge x1) = (0, 5 + 3) = (0, 8)$

3.1.2 Prefix operations

This kind of operation consists of only one expression and one operand, which makes them easier than the previous ones. Also, in this category there are only three different operands, which bounds are calculated as follows:

3.1.2.1 Subtraction

This operation denominated as "Sub", consists of a minus before the expression, and because of that, the minimum is -max, and the maximum is -min. As we want values between $[0, \text{prime})$, we will add $\text{prime}-1$ to both.

If $\text{Bounds}(e1) = (\text{min1}, \text{max1})$ then

$\text{Bounds}(-e1) = (-\text{max1} + \text{prime}-1, -\text{min1} + \text{prime}-1)$

3.1.2.2 Not

This operation denominated as "BoolNot" is used for Boolean expressions, and because of that, its bounds are easier as they are $[0, 1]$.

If $\text{Bounds}(e1) = (\text{min1}, \text{max1})$ then

$\text{Bounds}(! e1) = (0, 1)$

3.1.2.3 Complement

These binary operations convert all 1s to 0s and vice versa, therefore, the minimum will be 0, and the maximum will be twice the max values of the expression.

If $\text{Bounds}(e1) = (\text{min1}, \text{max1})$ then

$\text{Bounds}(\sim e1) = (0, \text{max1}*2)$

3.1.3 Switch operations

The switch operation works as an if else operation, so we will have two expressions and the context. To calculate the bounds of the switch operation, we will first calculate the bounds of both expressions, and then, we will keep the minimum value of both expressions' minimums, and the maximum value between both expressions' maximums.

If $\text{Bounds}(e1) = (\min1, \max1)$ and $\text{Bounds}(e2) = (\min2, \max2)$ then

$\text{Bounds}(\text{Switch: } e1, e2) = \{\min(\min1, \min2), \max(\max1, \max2)\}$

3.1.4 Variables

With the variables, we will receive an String and the context, and to obtain the bounds of the variables, we will just have to reach in the context if the variable already exists, and clone its bounds, or if it doesn't exist on the context, we just have to create its bounds which will be $[0, \text{prime})$.

3.1.5 Numbers

With the numbers, it will be much easier than any other operation, because its maximum and minimum bounds will have as value the given number.

3.1.6 Array in line

In the case of the array in line, we want to find the minimum bound between all the elements of the array, and the maximum bound of the same elements, so we will have to traverse the array trying to find those values.

As an example, suppose that $x[0] \in [0, 5]$, $x[1] \in [2, 7]$ and $x[2] \in [1, 4]$. Then:

$\text{Bounds}(x[]) = \{\min(0, 2, 1), \max(5, 7, 4)\} = (0, 7)$

3.1.7 Uniform array

A uniform array represents a collection of values that are all computed from a single expression. When computing the bounds of a uniform array, the key idea is that all its elements will have identical bounds, since they are generated from the same expression. For this reason, the analysis only needs to compute the bounds of that expression once.

Given an expression value, the bounds of the uniform array are defined as:

$\text{Bounds}(\text{UniformArray}(\text{value})) = \text{Bounds}(\text{value})$

3.2 Treat statements

In the previous section, we described how to compute the bounds of individual expressions. However, programs are not composed only of expressions, but also of statements that modify the state of the computation.

The goal of this part is to maintain updated bounds of all signals throughout the execution of the program. To do that, the analysis keeps a context, implemented as a hash map, where each variable is associated with its current bounds.

3.2.1 Treat conditionals

Conditional statements (if-then-else) require analyzing multiple possible execution paths. The analysis must consider both branches as potentially executable.

To do this, the analysis creates two copies of the current context: one for the if branch and one for the else branch. Each branch is then analyzed independently, updating its context based on the statements it contains.

Once both branches have been analyzed, the resulting contexts are merged. For each variable, if it appears in both contexts, the final bounds are computed by taking the minimum of the lower bounds and the maximum of the upper bounds:

$\text{min} = \min(\text{min if}, \text{min else}), \text{max} = \max(\text{max if}, \text{max else})$

If a variable appears only in one branch, its bounds will be transferred to the final context.

3.2.2 Treat while loops

While loops are more challenging to analyze, because the number of iterations is not known to us. This makes it difficult to precisely determine how the bounds of variables evolve during execution.

To address this, the analysis adopts a conservative approach. First, all the bounds are going to have a Boolean value indicating if they are loop-safe. This means that the value of those bounds is not affected by the number of iterations of the loop. This can happen with constants, Boolean operations, and some of the infix and prefix operations when all the operands are loop-safe.

Secondly, we are going to check which variables contain the loop, and from those variables, we are going to change the bounds of the loop-unsafe variables to no bounds $[0, \text{prime})$. In that way we make sure that the calculated bounds are reliable when going through a loop.

The loop-safe calculations can be helpful in optimization because in big templates it is very common to have loops, and we need to keep the bounds as small as possible. Therefore, searching for these loop-safe variables is the way to maintain the optimization process through the loop.

3.3 Bounds inheritance

Circom programs are usually composed of many templates that call each other, and they usually send values and expect a value in return.

Due to those values inherited from father templates, and the ones received from child templates, the optimization is not complete yet. We need some communication, so the templates know the bounds of the values that they are receiving and use them to calculate narrower bounds.

Communication consists in three steps:

1. We calculate bounds in each template considering only the statements.
2. For each template, we take the bounds of the received input variables from the father template and recalculate the bounds using the new information.
3. For each template, we send the output bounds to the father template, and recalculate the bounds one last time, now with all the information.

With these three steps, we have solved the issue of inheritance of bounds, and now we will have narrower bounds which will make the program more optimized.

Capítulo 4 - Testing

Testing is very important because we can measure the impact of the whole project in real situations.

To do that, I am going to use the Circomlib repository on GitHub, which provides examples of Circom code to test the functionality of the compiler. In my case I am going to test how much memory we could save with this optimization.

I am going to take some programs of the Circomlib, and based on the bounds that it gives me, I can relate those bounds to smaller data types such as Integers and Booleans. These smaller data types will make a difference in memory space and in execution time, as they are faster to process.

To show the results, I am going to use a table in which each row is going to reflect the data of one test program, and it will have the frequency of each kind of data size (Bool, Int or prime), and the percentage of memory usage reduction, taking into account that Bool's size is 1 byte, Int's size is 4 bytes and prime's size is 32 bytes.

File name	Number of signals	Bool	Int	prime	% of memory saved
Aliascheck_test	21	1	0	20	4.6%
Babyadd_tester	24	9	6	9	58.2%
Babycheck_test	12	6	6	0	92.2%
Binsub_test	23	3	1	19	16.4%
Escalarmulany_test	105	9	11	85	17.5
Escalarmulw4table	12	1	0	11	8%

File name	Number of signals	Bool	Int	prime	% of memory saved
Greaterqthan	25	2	1	22	11.3%
isequal	5	1	0	4	19.4%
IsZero	6	3	0	3	48.4%
lessthan	7	1	0	6	13.8%
Mimc_test	26	0	9	15	28%
Montgomeryadd	18	3	9	6	59.9%
Mux1_1	21	6	2	13	36%
Sum_test	25	2	1	22	11.3%

Table 4-1. Test results

As expected, the results vary depending on the program, and therefore the results of the optimization are more notable in some programs than in others.

In average we have save 30.4% in memory usage, which is an advantage from using numbers from 0 to the prime. But regardless of the memory usage, all these programs could also benefit from faster execution times, as smaller data types are easier to manage.

Capítulo 5 - Conclusions and future work

Conclusions

This project solves memory inefficiency within the Circom compiler. Because Circom treats all signals as maximum-width prime field elements, generated witness calculators carried unnecessary computational and memory overhead. The implementation of a bound analyzer allows the compiler to predict signal ranges through complex infix, prefix, and bitwise operations, as well as through program state changes like conditionals and loops.

The results validate the effectiveness of this optimization. By testing the bound's analyzer through some circuits from the Circomlib repository, it was proven that assigning appropriate primitive types, like booleans and integers, generates a performance gain. Across the test programs, memory consumption was reduced by an average of 30.4%, with highly constrained circuits like Babycheck_test achieving up to a 92.2% reduction in memory footprint. Consequently, replacing 32-byte types with 1-byte or 4-byte alternatives ensures that the generated C++ and Web Assembly code executes faster and scales better for large cryptographic applications.

Future work

While the bound's analyzer does a good job, there still is room for improvement. For example, the loop analysis could get some improvement to reduce the number of signals without bounds.

Also, the bound's analyzer could be implemented in the actual Circom compiler, so the actual compiler benefits from this optimization

BIBLIOGRAPHY

- [1] CIRCOM: A Robust and Scalable Language for Building Complex Zero-Knowledge Circuits upcommons.upc.edu
- [2] Circom 2 Documentation docs.circom.io
- [3] Circom ReadMe github.com/iden3/circom/blob/master/README.md
- [4] Rust's Strategic Advantage sysid.github.io/
- [5] Circomlib <https://github.com/iden3/circomlib>

